



LOG8100 - DEVSECOPS

Laboratoire 2

Tests de pénétration et Outils pour Intégration et Déploiement continus

Soumis par:

Équipe 12

Ian Alexis - 2065566

Ahmed Gafsi - 1621855

Feriel Tabbane - 1951648

Manel Ben Jemaa - 1871842

Soumis à : Antoine Boucher

24 Octobre 2024

AUTOMNE 2024

Introduction	3
Analyse de sécurité	3
Description du pipeline d'intégration continue (CI)	6
Outils et fonctionnalités	6
Description du pipeline de déploiement continu (CD)	7
Architecture de déploiement	8
Registre d'images	9
Infrastructure Cloud	9
Configuration des variables d'Environnement dans Azure Container Apps	10
Conclusion	10
Références	11

Introduction

L'application DVNA (Damn Vulnerable NodeJS Application) est utilisée comme plateforme pour tester des outils de tests de pénétration et d'évaluation de la sécurité, notamment OWASP ZAP pour identifier des vulnérabilités. Les objectifs principaux de ce laboratoire sont de développer et de configurer un pipeline CI/CD sécurisé en utilisant des outils tels que GitHub Actions, Docker, Snyk, CodeQL, Dependabot et Azure. Les prochaines sections abordent l'analyse de l'application, la description du pipeline d'intégration continue et de déploiement continue, et la conclusion qui résume les résultats et les recommandations pour améliorer la sécurité de l'application.

Analyse de sécurité

- Résultats des test de pénétration:

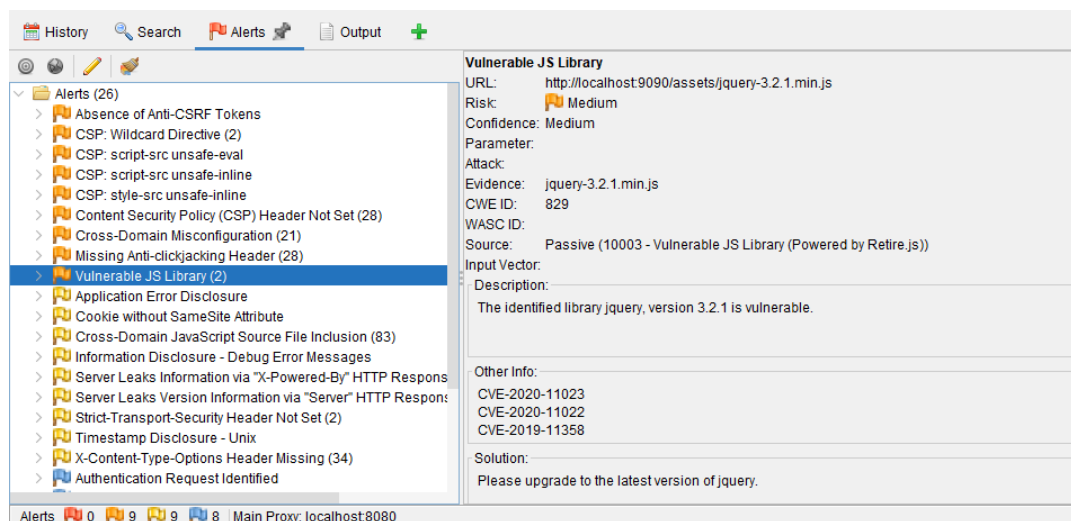


Figure 1: Vulnérabilités trouvées dans l'application DVNA avec OWASP ZAP

Dans l'image ci-dessus, nous pouvons voir les résultats des tests de pénétration effectués avec l'outil OWASP ZAP en navigant sur l'application DVNA. Après l'exécution des tests, ZAP a trouvé 26 alertes différentes, dont **9 alertes ayant un risque medium, 9 ayant un risque bas et 8 alertes de type informatives**. Il est aussi possible de trouver les mêmes résultats et possiblement plus d'alertes en utilisant l'image Docker de ZAP pour effectuer les tests d'intrusion.

- Description des vulnérabilités identifiées et de leur gravité:

- a. Librairie JavaScript vulnérable
 - Risque: Medium
 - La librairie jQuery version 3.2.1 est vénérable. Les vulnérabilités des versions de jQuery qui précèdent la version 3.5.0 peuvent permettre des attaques de type *Cross-Site Scripting (XSS)*, où un attaquant injecte du code malveillant pour voler des données d'utilisateur, manipuler l'interface ou exécuter des actions non autorisées.
- b. Cookie sans l'attribut SameSite
 - Risque: Bas
 - Un cookie a été défini sans l'attribut *SameSite*, ce qui signifie que ce cookie peut être envoyé lors d'une requête provenant d'un site différent. L'attribut *SameSite* est une mesure de sécurité efficace contre plusieurs types d'attaques, telles que la *Cross-Site Request Forgery (CSRF)*, qui consiste à envoyer des requêtes non autorisées d'un site externe, l'inclusion de scripts intersites (*cross-site script inclusion*), et les attaques par temporisation (*timing attacks*). Sans l'attribut *SameSite*, les navigateurs peuvent envoyer ce cookie avec des requêtes entre différents sites, ce qui augmente les risques de vol d'identité et d'exécution d'actions non souhaitées sur le site visité. L'ajout de cet attribut, configuré à des valeurs telles que *SameSite=Strict* ou *SameSite=Lax*, aide à limiter l'utilisation des cookies à l'intérieur du même site, ce qui renforce la sécurité des sessions utilisateur.
- c. Absence de jetons Anti-CSRF
 - Risque: Medium
 - Aucun jeton Anti-CSRF n'a été détecté dans le formulaire de soumission HTML. Une attaque CSRF est donc possible. La cause principale de cette vulnérabilité est l'usage de fonctionnalités d'application utilisant des URL ou des actions de formulaire prévisibles de manière répétitive.
- Propositions de stratégies de mitigation pour chaque vulnérabilité
 - a. Librairie JavaScript vulnérable
 - La solution pour corriger cette vulnérabilité est de changer la version de jQuery à une version plus récente.
 - b. Cookie sans l'attribut SameSite

- La solution pour corriger cette vulnérabilité est d'ajouter l'attribut *SameSite* pour tous les cookies et s'assurer que la valeur de cet attribut soit idéalement *Strict* ou *Lax* au minimum.
- c. Absence de jetons Anti-CSRF
- Il y a plusieurs solutions pour corriger cette vulnérabilité. Une des solutions possible est de vérifier l'en-tête HTTP *Referer* pour voir si la requête provient de la page attendue. Une autre solution possible est d'utiliser une bibliothèque ou un framework vérifié qui empêche l'apparition de cette faiblesse ou qui fournit des structures facilitant la prévention de cette vulnérabilité. Par exemple, des packages anti-CSRF tels que OWASP CSRFGuard.

Description du pipeline d'intégration continue (CI)

Outils et fonctionnalités

Suite à l'identification des vulnérabilités, nous avons mis en place un pipeline d'intégration continue pour automatiser la détection et la prévention des problèmes de sécurité.

1. Gestion des dépendances :
 - Outil utilisé : Dependabot
 - Fonction : Vérifie quotidiennement les mises à jour pour les dépendances GitHub Actions, npm, et Docker
2. Tests de sécurité :
 - Outil principal : Snyk
 - Fonctions :
 - Analyse les vulnérabilités dans le code et les dépendances
 - Génère des rapports au format JSON et SARIF
 - Crée un rapport HTML pour une visualisation facile des résultats
 - Outil principal : CodeQL
 - Fonctions :
 - Analyse le code source pour identifier les vulnérabilités potentielles dans l'application
 - Génère des alertes sur les failles de sécurité comme les injections SQL ou les XSS, et il offre des solutions pour les corriger.
3. Déploiement de la documentation :
 - Plateforme : GitHub Pages
 - Fonction : Déploie automatiquement le contenu du dossier ./docs sur GitHub Pages
4. Publication d'image Docker :
 - Plateforme : Docker Hub
 - Fonction : Construit et pousse l'image Docker de l'application vers Docker Hub

Diagramme d'activité des étapes de notre pipeline :

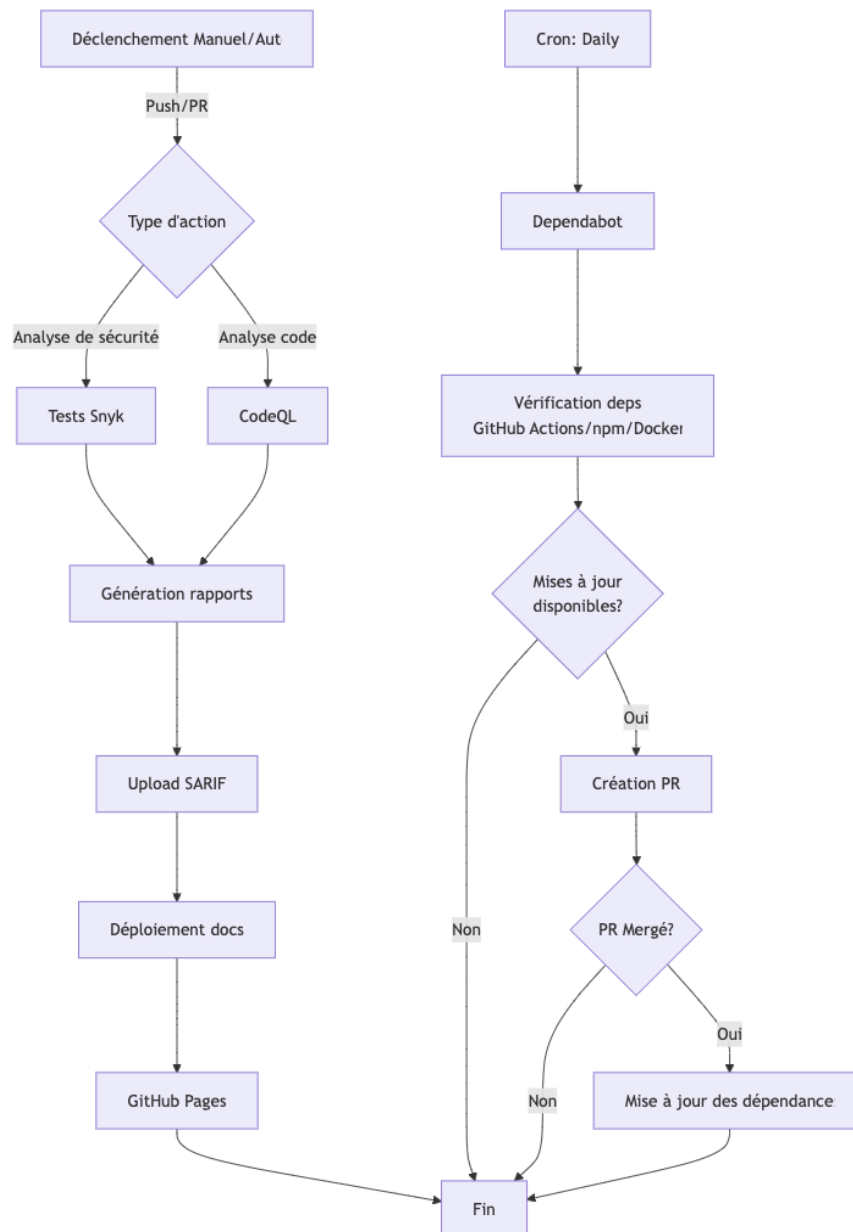


Figure 2: Diagramme d'activité du pipeline CI

Le diagramme illustre les principales étapes de votre pipeline CI/CD, y compris les tests de sécurité avec Snyk, le déploiement sur GitHub Pages, et la publication d'images Docker. Il montre également les différents points de déclenchement de la pipeline et les flux d'exécution conditionnels basés sur le type d'événement ou les étapes optionnelles.

Description du pipeline de déploiement continu (CD)

Architecture de déploiement

Après avoir établi les bases de l'intégration continue, nous avons développé une pipeline de déploiement continu qui nous permet une livraison sécurisée et automatisée de l'application.

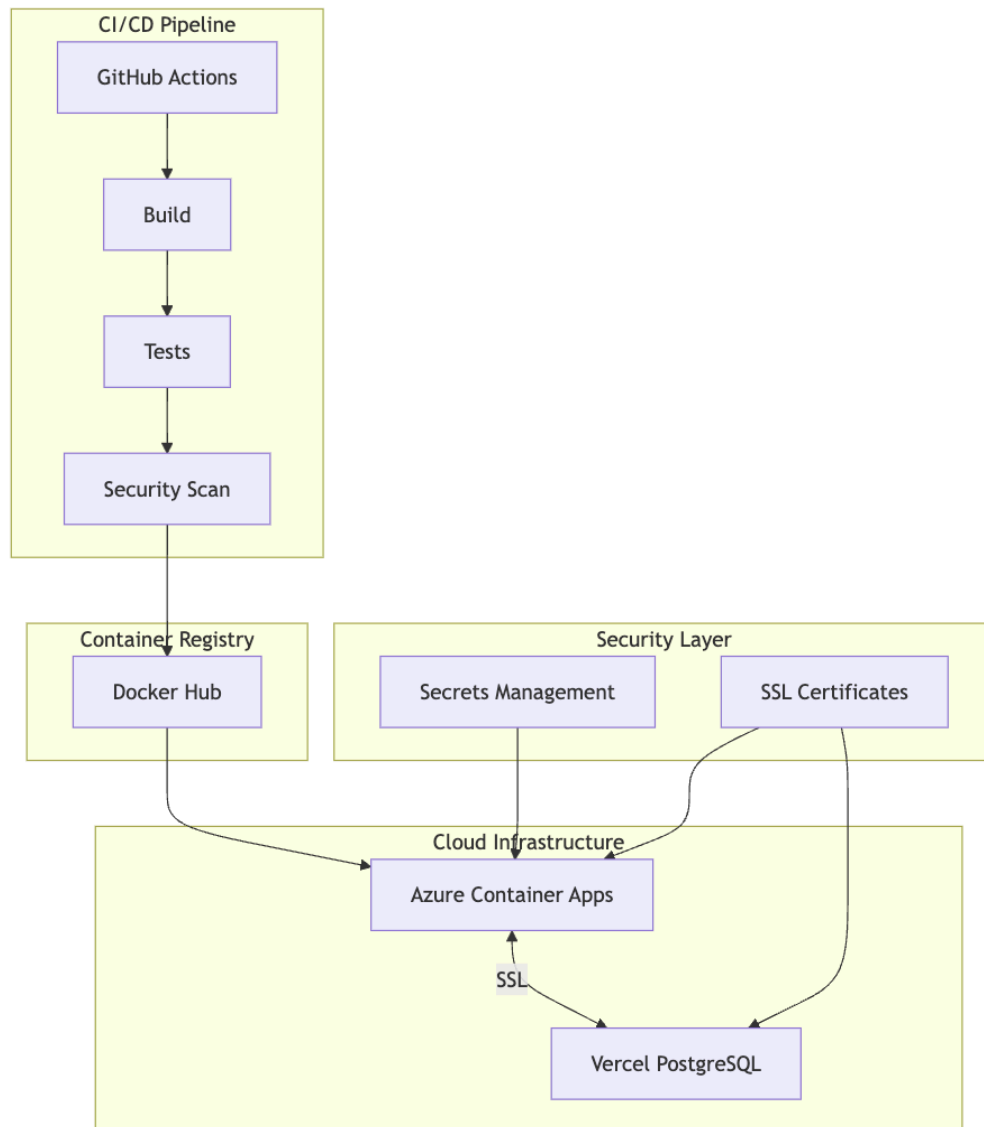


Figure 3: Diagramme d'architecture de déploiement

Registre d'images

1. Processus de déploiement :

- Le déploiement est déclenché automatiquement sur push vers main ou manuellement via **workflow_dispatch**
- Utilisation de trois plateformes de déploiement :
 - Docker Hub pour le stockage des images
 - Vercel pour la base de données PostgreSQL
 - Azure pour l'hébergement de l'application

2. Étapes du processus :

a. Publication de l'image Docker :

- Checkout du code
- Login à Docker Hub
- Build et tag de l'image
- Push vers Docker Hub
- Nettoyage des images locales

b. Déploiement Azure :

- Authentication Azure
- Déploiement de l'application conteneurisée
- Configuration de la connexion à la base de données

Infrastructure Cloud

1. Application :

- Hébergée sur Azure Container Apps
- Accessible via l'URL fournie par Azure après le déploiement

2. Base de données :

- Hébergée sur Vercel
- Configuration :
 - Host: ep-noisy-queen-a4fnpu0f-pooler.us-east-1.aws.neon.tech
 - Port: 5432
 - Database: verceldb

3. Registry Docker :

- Images stockées sur Docker Hub
- Repository: ianou27/log8100_tp2
- Tags: latest

Configuration des variables d'Environnement dans Azure Container Apps

La mise en place de l'infrastructure cloud nécessite une attention particulière à la configuration des variables d'environnement pour garantir un fonctionnement sécurisé de l'application.

Lors de la configuration des variables d'environnement pour Azure Container Apps, nous avons initialement rencontré des difficultés, car les variables d'environnement définies dans le code, comme **databaseurl**, ne fonctionnaient pas comme prévu. Pour résoudre ce problème, nous avons commencé par ajouter la variable **databaseurl** dans les secrets du conteneur TP2 sur Azure. De plus, nous avons modifié le fichier d'environnement sur GitHub pour y inclure la configuration **ssl:{require:true}**. Nous nous sommes aussi assurés de veiller à ce que toutes les lettres soient en minuscules, pour le nom de la variable d'environnement, afin de respecter les contraintes de format imposées par Azure.

Après ces ajustements, nous avons accédé directement à la configuration des conteneurs sur Azure. En cliquant sur "**edit and deploy**", nous avons pu accéder à l'option des variables d'environnement. À cet endroit, nous avons ajouté manuellement la variable **databaseurl** ensuite nous avons sélectionné l'option "**reference a secret**" pour l'associer à la valeur du secret correspondant déjà configuré. Ces étapes nous ont permis de faire fonctionner correctement la variable d'environnement et de garantir une connexion sécurisée à la base de données, avec l'activation de l'option SSL.

Conclusion

Ce projet nous a permis d'identifier des vulnérabilités critiques telles que des failles XSS et l'absence de jetons CSRF à l'aide d'outils comme OWASP ZAP, CodeQL et Snyk, tout en mettant en place des pipelines CI/CD automatisant les tests de sécurité et les déploiements. Cela a garanti des mises à jour rapides et sûres de l'application, renforçant ainsi sa posture de sécurité. Il est recommandé de mettre à jour régulièrement les dépendances, d'implémenter des jetons CSRF, d'intégrer des tests de pénétration avancés, d'adopter des pratiques de **Security as Code**, et de renforcer la gestion des secrets.

L'application continue de ces pratiques **DevSecOps** améliorera non seulement la sécurité, mais aussi l'efficacité du cycle de développement et de déploiement.

Références

<https://github.com/lanou27/log8100-tp2>